Sliding Puzzle Project — Development Log

Day 2 — Thursday, July 30, 2026

Goal for the project

Build four Python puzzle projects (a playable sliding tile puzzle, a solver for it, a playable 3D Rubik's cube, and a solver for that) as a learning exercise. The user writes all code themselves; guidance is conceptual, not code handed over directly.

Today's focus

Stage 1 of the sliding puzzle: choosing a data structure to represent the board, and building the two functions needed to create and display a solved board before any graphics or movement logic exists.

What happened

- Re-established project continuity after losing track of a prior chat session (July 28) that contained no actual project work — confirmed via conversation search that the only real prior session was Day 1 (environment setup). Set up a standing memory note so future sessions in this project carry context automatically without needing the PDF re-uploaded every time.
- Worked through the choice of data structure to hold the puzzle's state (solved or shuffled). Considered and rejected several alternatives before settling on one:
- List of tuples/dictionary pairing a flat coordinate list (e.g. 11, 12, 13...) against a tile list — rejected: no direct lookup by position, and the coordinate scheme (tens = row, ones = column) breaks down at board sizes with two-digit rows/columns.
- Separate row-array and column-array — rejected: still needs a third structure to tie a row and column together to a tile, more bookkeeping than a single 2D structure.
- Flat list with an index-to-(row,col) conversion formula (row = index // cols, col = index % cols) — workable, but requires that conversion on nearly every operation (finding the blank, checking legal moves, printing), even when wrapped in a helper function; edge/wraparound checks (e.g. can't move left off the start of a row) still have to be handled explicitly regardless of how the formula is packaged.
- Landed on: a dictionary with (row, col) tuples as keys and tile values, e.g. {(0,0): 1, (0,1): 2, ...}, built from an initial flat list. Chosen over a plain list of lists because it gives direct coordinate-keyed lookup, which the user wants for later stages; both structures are mutable, so both support in-place tile moves.

Wrote and debugged four functions in board.py, in this order:

- create_solved_board() — prints a menu (1: 3x3, 2: 3x4, 3: 4x4), takes the choice via input(), and returns the correct flat tile list (State) for that size, with '_' as the blank in the last position.
- num_of_columns(State) — derives the column count from len(State) (9 → 3, 12 → 4, 16 → 4), avoiding the need to return a second value out of create_solved_board().
- num_of_rows(State) — derives row count as len(State) // num_of_columns(State).
- coordinates(State, index) — converts a flat index into a (row, col) tuple using num_of_columns internally.
- create_grid_dictionary(State) — loops over State with enumerate(), builds the {(row,col): tile} dictionary using coordinates().

- print_grid(State) — loops over State with enumerate(); prints each tile with end=" " to keep a row on one line, and switches to a plain print() (default newline) whenever (index + 1) % number_of_columns == 0, i.e. at the end of each row.

Confirmed create_solved_board() + create_grid_dictionary() together produce a correct dictionary for a 3x3 board:

```
{(0, 0): 1, (0, 1): 2, (0, 2): 3, (1, 0): 4, (1, 1): 5, (1, 2): 6, (2, 0): 7, (2, 1): 8, (2, 2): '_'}
```

— correct shape, correct order, blank in the correct position.

print_grid() reached a final, correct version producing the expected readable grid layout (e.g. "1 2 3" / "4 5 6" / "7 8 _" for a 3x3 board); the last run to confirm actual printed output on-screen was still pending at end of session.

Concepts & Lessons Learned (study notes)

Choosing a data structure by testing operations, not just storage

The right way to compare candidate data structures isn't just "can it hold the data" — almost anything can. The real test is walking through the actual operations you'll need (look up a value by position, change a value, check a neighbour, iterate in order) and seeing which structure needs the fewest extra steps for each one. A structure that technically works but needs constant index math or manual searching is a real cost paid on every future stage, not a one-time setup cost.

Local scope and UnboundLocalError

A variable created inside a function only exists while that function is running (its "local scope") — nothing outside the function can see or use it once it returns, unless the value is explicitly returned. UnboundLocalError specifically means: some code path tried to use a variable that was supposed to be created inside the function, but the branch that would have created it never ran. The traceback points at the line that tried to use the variable, but the actual bug is upstream — in whatever condition decided whether the assignment happened at all. In this project the root cause was a type mismatch (see below): comparing a string to an integer meant no branch of an if/elif chain ever matched, so the variable was never assigned.

input() always returns a string

Python's input() returns whatever the user typed as a string, even if it looks like a number. Comparing that directly to an integer (e.g. size == 1 when size holds "1") is always False, silently — no error is raised, the condition just never matches. This was the root cause of the UnboundLocalError above. General checklist when a variable "isn't changing" as expected: (1) type mismatch in the condition, especially after input(); (2) comparing against a value the user isn't actually prompted to type; (3) indentation putting code inside/outside the wrong block; (4) an unconditional reassignment later in the function overwriting the intended value; (5) checking the variable somewhere outside the scope where it was created.

Returning multiple values from a function

A function can return more than one value at once by separating them with a comma (return a, b), and the caller unpacks them the same way (x, y = my_function()). This came up when create_grid_dictionary needed both State and number_of_columns, but create_solved_board only returned one. The alternative used here — deriving number_of_columns separately from len(State) via a dedicated function — is a valid way to sidestep returning multiple values, though it's slightly more fragile in general: it relies on tile-count being unique per board size, which happens to hold for 3x3/3x4/4x4 but wouldn't distinguish two different shapes that happen to share the same tile count.

The % (modulo) operator and getting operand order right

a % b gives the remainder after dividing a by b. Order matters: when the first operand is smaller than the second (e.g. 3 % 6), the result is just the first operand itself, and can basically never reach 0 except by coincidence — which silently breaks any condition relying on "== 0" to detect a boundary. This surfaced when checking for end-of-row while printing: number_of_columns % (index + 1) == 0 only worked for the first row, since the expression needs the growing value (index + 1) on the left and the fixed value (number_of_columns) on the right — (index + 1) %

number_of_columns == 0 — to correctly land on 0 at every row boundary (indices 2, 5, 8, ... for 3 columns), not just the first.

print()'s end parameter

print() appends a newline character after its output by default, controlled by a keyword parameter end, which defaults to "\n". Passing end=" " makes consecutive print() calls continue on the same line instead of starting a new one — the mechanism used to print a full board row on one line, before calling a plain print() (default end) once per row to move to the next line.

Debugging by hand-tracing a condition

When a condition's boolean logic is in doubt, the reliable way to check it is to manually work out its value for every relevant input in a small table (e.g. index 0 through 8) rather than guessing or eyeballing the code. This caught two separate bugs in the end-of-row detection logic that weren't obvious from reading the code alone.

Status at end of day

Stage 1 (board representation, no graphics) is functionally complete: board state is stored as a {(row,col): tile} dictionary built from a flat list, with working functions to build a solved board for a chosen size, derive row/column counts, convert indices to coordinates, build the dictionary, and print the board as a readable grid. Final on-screen confirmation of print_grid()'s output was the last pending check at end of session.

Next up

Confirm print_grid() output looks correct for all three board sizes (3x3, 3x4, 4x4), then move to Stage 2 — core movement logic: find_blank(), legal_moves(), apply_move(board, direction), and is_solved(board), tested via a text-based loop in the terminal before any graphics are introduced.